Android Dev Summit

Building Agentic Apps



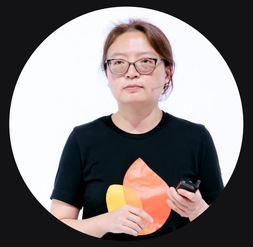
Kateryna Semenova
Android DevRel



Ramya Raghavan
Gemini Partner Engineering



Jolanda Verhoef
Android DevRel



Cynthia Jiang
Firebase Engineer

Logistics & awards

Sprint Logistics

2 Hours Coding
Rapid prototyping on existing apps

Test Devices Provided
Pixel 11 per table

On-Site Mentors
DevRel and Partner Eng support in room

1 Hour Demos
Live presentations & judging on stage

Evaluation Criteria

Technical Integration
Robust use of AppFunctions, MCP servers, ML Kit, or Firebase.

User Impact & Value
Solves real user friction with delightful, reliable agent behavior.

Creativity & Innovation
Novel agentic concepts, creative UX.

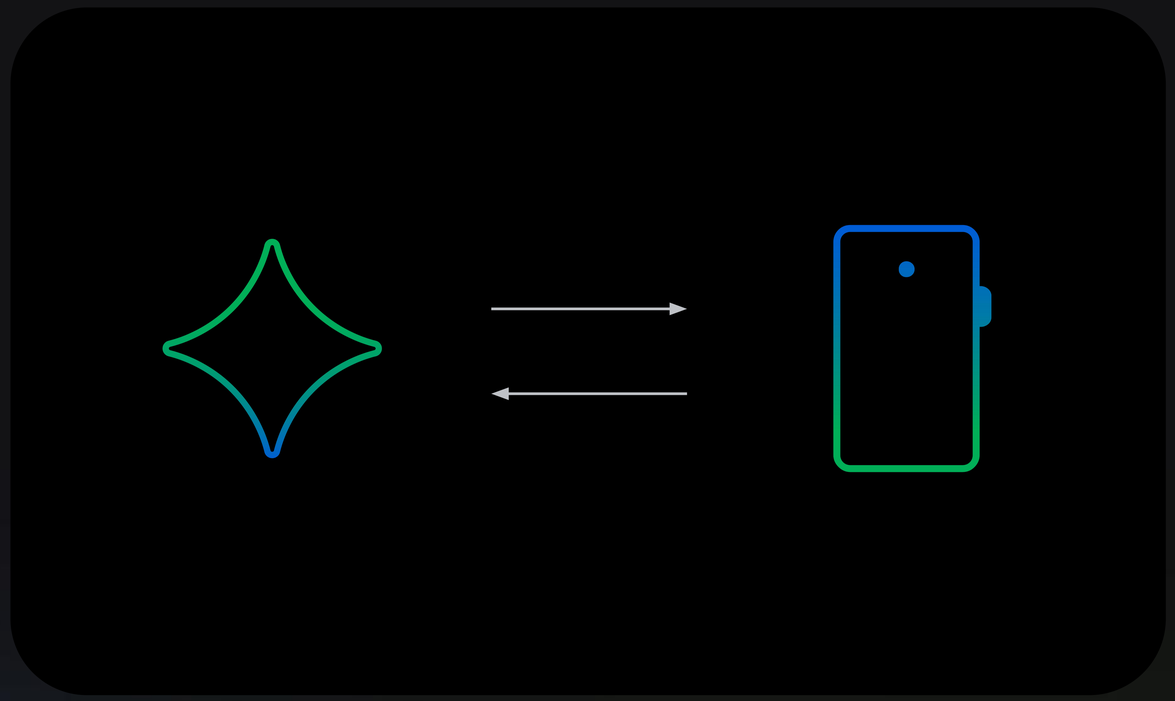
Awards & Prizes

Best Overall App
1 team: 4× Pixel 11 + Google AI Ultra

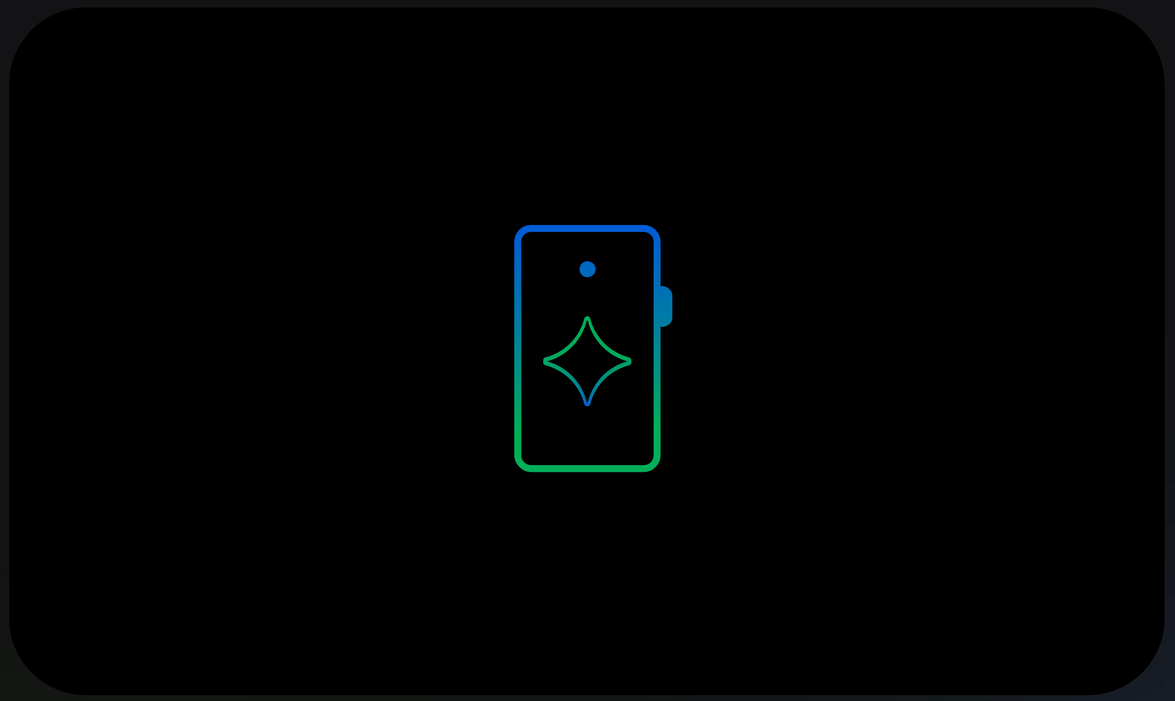
3 Sub-Category Winners
Google AI Ultra for each category's winning team

Participation Prize

Hackathon

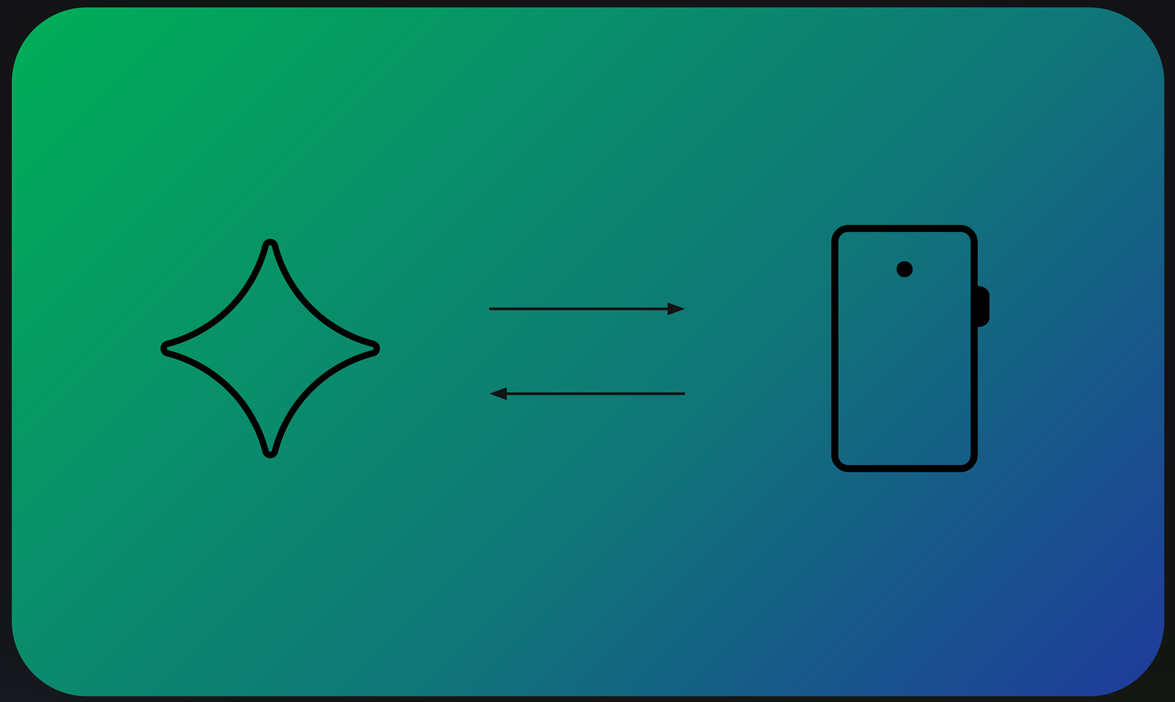


Integrate with the Gemini agent
via the Connected Apps Platform

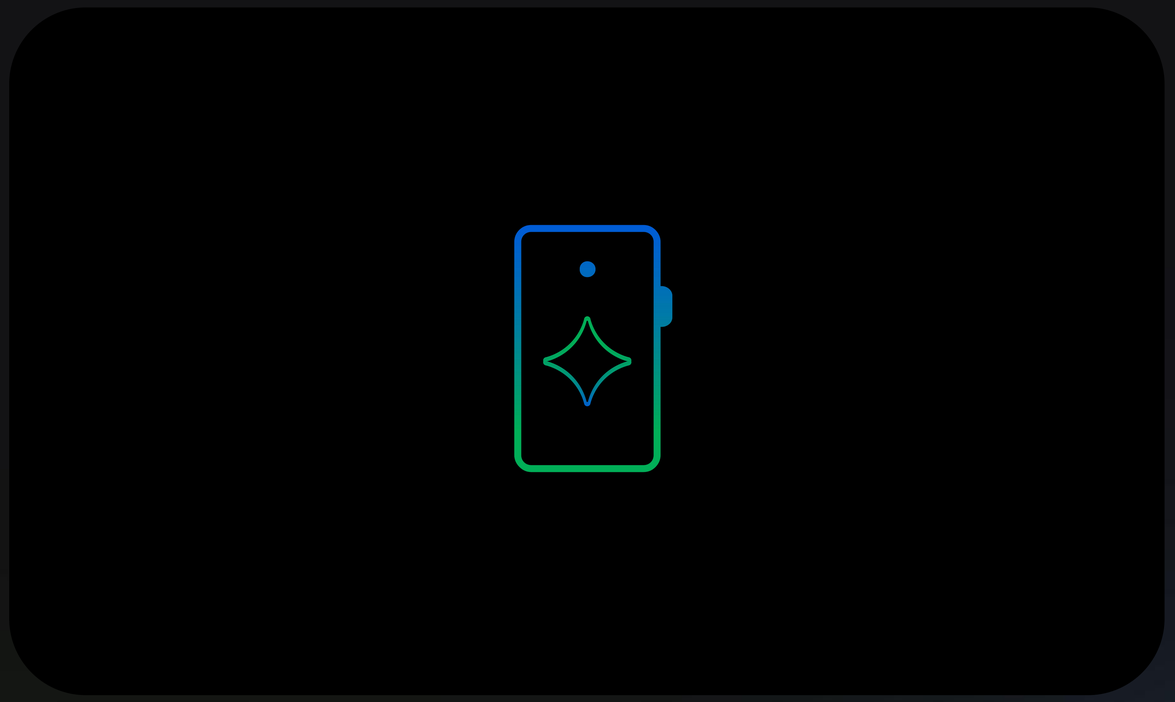


Build in-app agents using
ML Kit GenAI and Firebase AI Logic

Hackathon



Integrate with the Gemini app
via the Connected Apps Platform



Build in-app agents using
ML Kit GenAI and Firebase AI Logic

Integrate with the Gemini app

MCP server

Best for cross platform integrations (Android, iOS and Web) using your existing server-hosted endpoints

Use the open Model Context Protocol (MCP) spec with OAuth 2.1 and Google account linking.

Use MCP Inspector for rapid server testing nd request/response verification.

Android MCP (AppFunctions)

Best for local on-device data access, ex: local app data or state that can't be accessed via cloud.

Share Android app functionality directly via the Jetpack AppFunctions SDK. Use Skills file for quick prototyping.

Use the Agent sample app to test AppFunctions invocations. https://github.com/android/appfunctions

Test end-to-end with Gemini agent

MCP server

Access via dev mode:

Step 1

Enable Developer Mode

Open Gemini -> Go to Settings -> Developer Mode

Step 2

Create a custom MCP app

Add custom app -> Test with different prompts -> Debug

Android MCP (AppFunctions)

Access via test accounts:

Step 1

Request Test Account Access

Reach out to staff members for test account access. You would need to fill in the access form.

Step 2

Receive Invitation

Accept the invitation via email. Create a connected app in Play Console and test with Gemini app.

Best practices cheat sheet

Remote MCP server

1 Integration Standards & Auth Options

Structure & UX: Atomic verb_noun tools, flat intent-driven schemas & errors, /v1 path versioning, and MCP Apps UI.

Auth Options: Choose No Auth, Machine-to-Machine (Client Credentials), or User-Delegated OAuth 2.1—via Pre-registered Client ID/Secret or .well-known Dynamic Client Registration / CIMD.

2 Platform Policy (Static Schemas & Safety)

No dynamic or runtime-fetched schemas; require ToolAnnotations for user confirmations on write & sensitive actions.

3 Sizing, Payloads & Transport

Aim for 4–15 outcome-oriented tools (max 1–2 calls per journey). Pre-aggregate human-readable fields (names, dates, venue, price) over Streamable HTTP only.

Android MCP (AppFunctions)

1 When to Use It (Android 16 / API 36+)

On-device actions via Jetpack androidx.appfunctions (@AppFunction) where you want local app state, offline capability, or zero backend overhead.

2 KDoc Is Your Prompt (isDescribedByKDoc = true)

Write agent-centric KDocs with explicit workflow hints, and put inline KDocs on every @AppFunctionSerializable property (KSP ignores @property tags!).

3 Pro Tip for Coding Agents & Testing

Run android skills add appfunctions to equip your AI coding assistant, and test locally with adb shell cmd app_function execute-app-function.

Need help during the sprint? Gemini Partner Engineering and DevRel mentors are in the room — raise your hand anytime!

Best practice 1: Provide explicit server instructions

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP(
    name="ADS-MCP",
    instructions=(
        "A Model Context Protocol server providing event "
        "overview, schedules, sessions, speakers, and "
        "sandbox demos for the Android Developer Summit "
    ),
)
```

Best practice 2:
Add tool annotations for safety & control

```
@mcp.tool(
    title="List Sessions",

    annotations=ToolAnnotations(
        readOnlyHint=True,
        destructiveHint=False,
    ),

    meta={ … },
)
def list_sessions( … )
…
```

Best practice 3:
Strongly typed schemas & semantic docstrings

```
def list_sessions(
    track: Optional[str] = None,
    location: Optional[str] = None,
) -> List[Dict[str, Any]]:
    """Lists all sessions, optionally filtered by track
    (e.g. 'Tools and performance', 'Apps Foundation',
    'AI experiences', 'Android XR') or location
    (e.g. 'MTV', 'Mountain View', 'London', 'LON').
    """

    …
```

Developer resources

MCP server

Integrate your services cross-platform via remote MCP server.

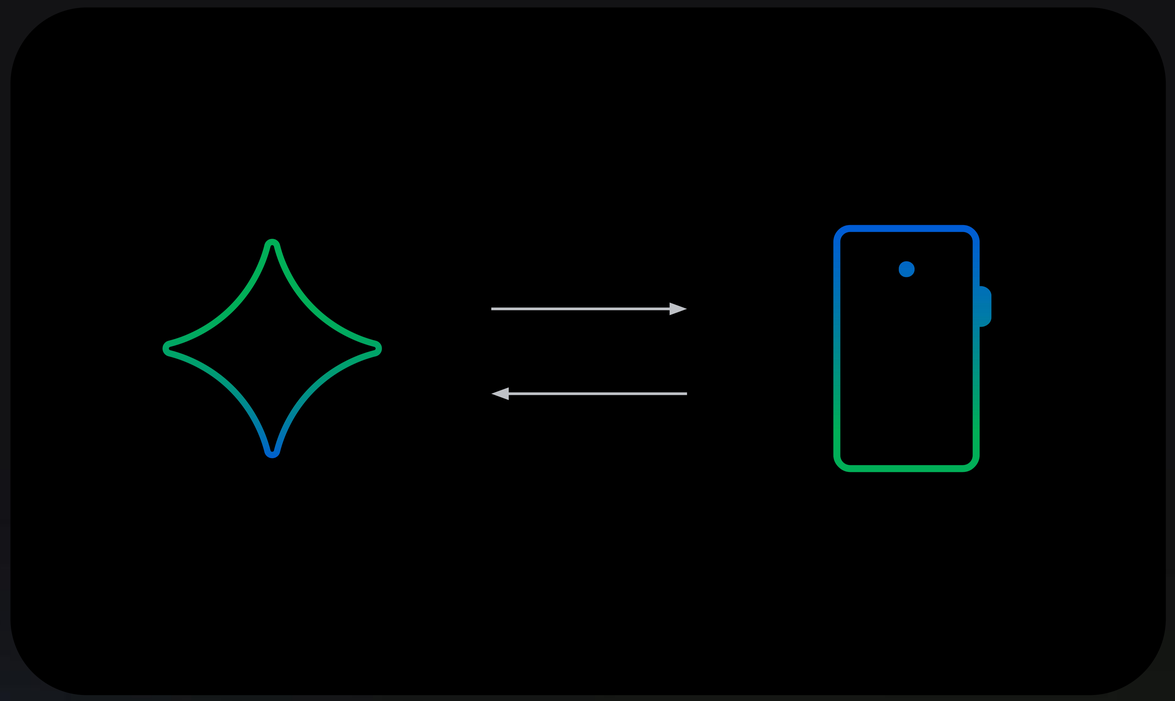
developers.google.com/connected-apps

Android MCP

Build on-device Android app tools with AppFunctions.

developer.android.com/ai/appfunctions

Hackathon

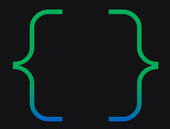


Integrate with the Gemini agent
via the Connected Apps Platform



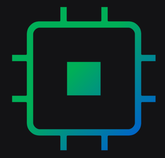
Build in-app agents using
ML Kit GenAI and Firebase AI Logic

Empowered by ML Kit GenAI and AICore, build in-app agents that run fully on-device



AppFunctions

define the agent's tools



Agent

holds the agent config



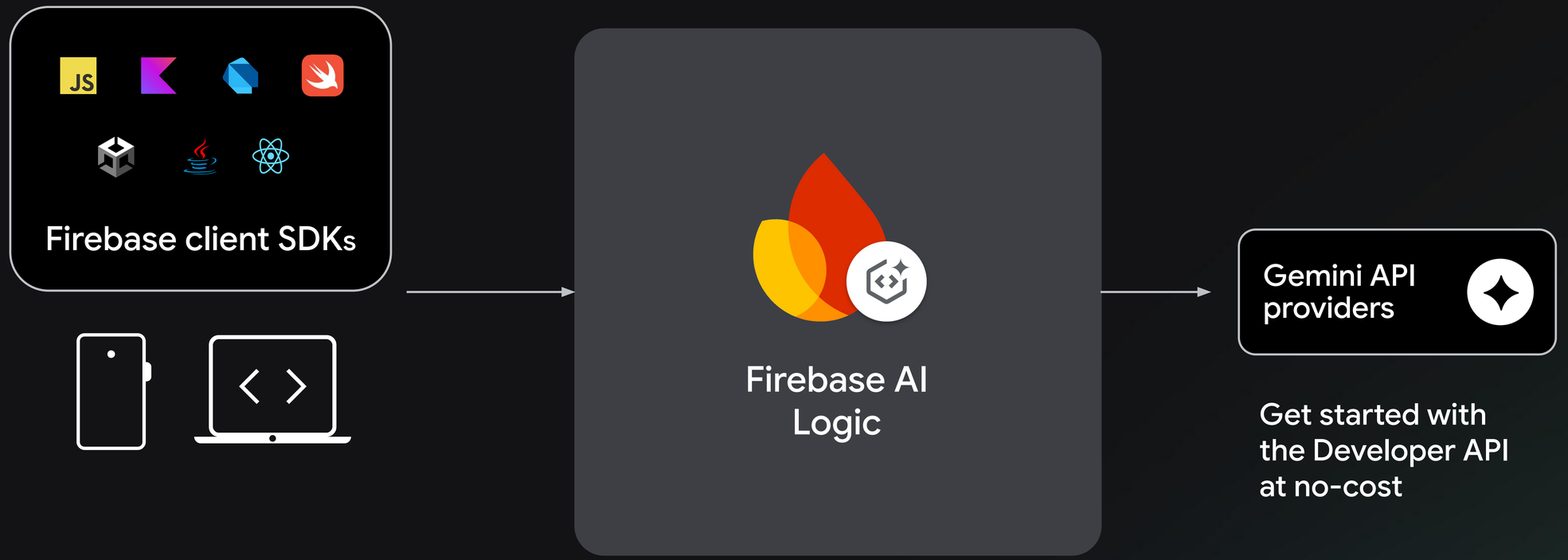
AgentRunner

manages execution sessions

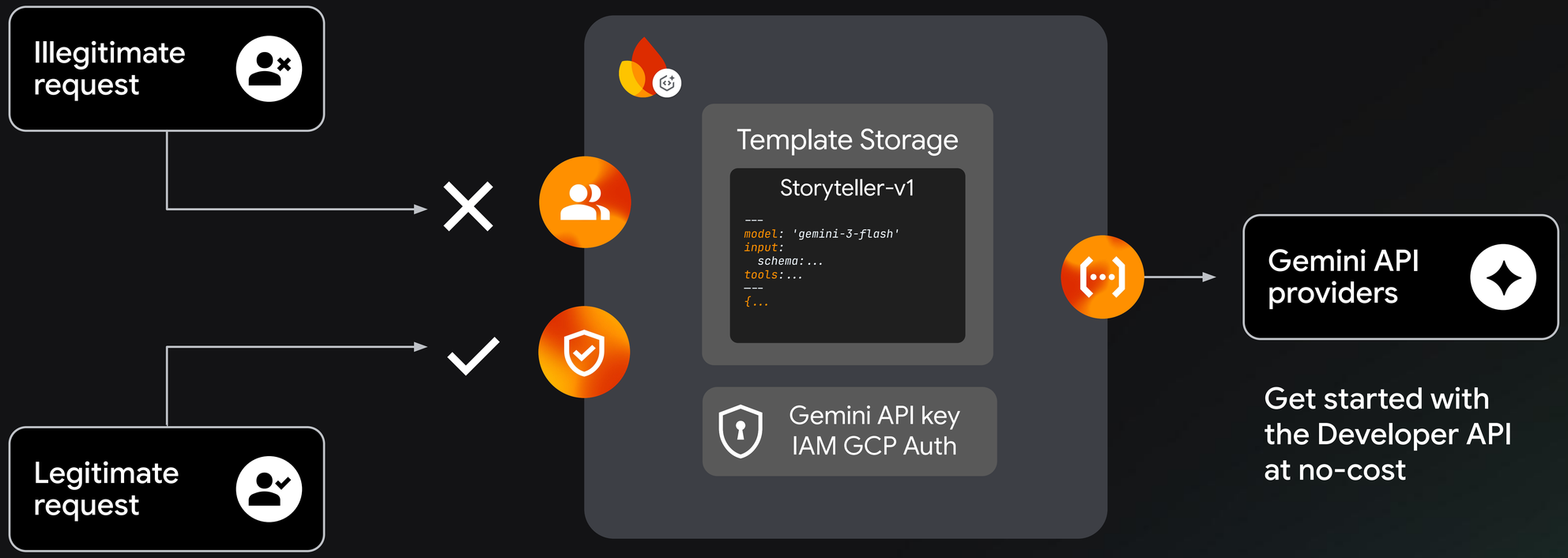


AgentEvent

emitted during execution



No need to deploy to or manage a server/backend (e.g., Cloud Run)



No need to deploy to or manage a server/backend (e.g., Cloud Run)

Developer resources

On-device agents

Run generative AI models locally on-device with low latency using ML Kit GenAI.

developers.google.com/ml-kit/genai

Cloud agents

Power scalable backend AI logic and cloud Gemini capabilities with Firebase.

firebase.google.com/docs/ai-logic